

Лабораторная работа №7. Компилятор GCC. Отладчик GDB. Отладчик DDD

Цель: изучение способов компиляции программ с помощью компилятора GCC и способов отладки программ с помощью отладчиков GDB и DDD.

ЗАДАНИЕ

1. Изучите главу 8 теоретического курса «Компилятор GCC» и главу 10 теоретического курса «Отладчики в Linux».
2. При необходимости установите пакеты gcc, make, gdb, ddd.

```
root@Guzovski:~# apt-get install -y gcc make gdb ddd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gcc is already the newest version (4:10.2.1-1).
gcc set to manually installed.
make is already the newest version (4.3-4.1).
make set to manually installed.
The following additional packages will be installed:
  libbabeltrace1 libboost-regex1.74.0 libc6-dbg libdebuginfod1 libipt2
  libmotif-common libsource-highlight-common libsource-highlight4v5 libxm
Suggested packages:
  ddd-doc pydb info qlibc-doc xterm gnuplot cups-bsd | lpr gdb-doc gdbser
```

3. Выполните на рабочем компьютере все практические примеры из данных глав.
4. Создайте на компьютере следующие файлы исходного кода программ на языке программирования C:

```

root@Guzovski:~# mkdir lab_c && cd lab_c
root@Guzovski:~/lab_c# cat << 'EOF' > main.c
#include <stdio.h>
#include <stdlib.h>
#include "list.h"
#include "matrix.h"

int main(int argc, char* argv[])
{
    printf("Пример программы на языке C\n\n");
    ListFunction();
    MatrixFunction();
    return 0;
}
EOF
root@Guzovski:~/lab_c# cat << 'EOF' > list.h
void ListFunction();
EOF
root@Guzovski:~/lab_c# cat << 'EOF' > list.c
#include <stdio.h>
#include <stdlib.h>
#include "list.h"

struct ListItem {
    int Value;
    struct ListItem* Next;
};

```

Файл main.c:

```

#include <stdlib.h>
#include "list.h"
#include "matrix.h"
int main( int argc, char* argv[] ){
    printf("Пример программы на языке C\n\n");
    ListFunction();
    MatrixFunction();
    return 0;
}

```

Файл list.h:

```

void ListFunction();

```

Файл list.c:

```

#include <stdlib.h>
#include "list.h"

struct ListItem {
    int Value;
    struct ListItem* Next;
};

```

```

struct ListItem* Root;

void AddNode( int NewValue ){
    struct ListItem* Item;
    Item = (struct ListItem*) malloc( sizeof(struct ListItem) );
    Item->Value = NewValue;
    Item->Next = Root;
    Root = Item;
}

void CreateList(){
    printf(" Создание списка.\n");
    Root = NULL;
    AddNode( 5 );
}

```

```

    AddNode( 4 );
    AddNode( 3 );
    AddNode( 2 );
    AddNode( 1 );
}

void ShowList(){
    struct ListItem* Item;
    printf(" Просмотр списка.\n");
    Item = Root;
    while (Item != NULL) {
        printf("( %d) , ", Item->Value);
        Item = Item->Next;
    }
    printf("\n");
}

void DestroyList(){
    struct ListItem *Item, *Item2;
    printf(" Удаление списка.\n");
    Item = Root;
    while (Item != NULL) {
        Item2 = Item->Next;
        free(Item);
        Item = Item2;
    }
    Root = NULL;
}

void ListFunction(){
    printf("Демонстрация работы со списком.\n");
    CreateList();
    ShowList();
    DestroyList();
    printf("\n");
}

```

Файл matrix.h:

```
void MatrixFunction();
```

Файл matrix.c:

```

#include <stdlib.h>
#include <math.h>
#include "matrix.h"

float ValMatr[30];

void MatrixFunction(){
    int I;
    printf("Запущен пример с матрицей.\n");
    for (I=0; I<29; I++){
        ValMatr[I] = 20*sin(I/10.0);
    }
}

```

```
    printf(" %d.\n",ValMatr[0]);
}
```

5. Выполните компиляцию единого исполняемого файла из трех файлов исходного кода. При компиляции нужно указать, что используется библиотека `libm.a`. Назовите полученный в результате исполняемый файл `testfile.out` и убедитесь в его работоспособности.

```
root@Guzovski:~/lab_c# gcc main.c list.c matrix.c -o testfile.out -lm
root@Guzovski:~/lab_c# ls
./testfile.out
list.c list.h main.c matrix.c matrix.h testfile.out
Пример программы на языке C
```

Демонстрация работы со списком.

Создание списка.

Просмотр списка.

(1), (2), (3), (4), (5),

Удаление списка.

Запущен пример с матрицей.

0.000000.

```
root@Guzovski:~/lab_c#
```

6. Выполните компиляцию статической библиотеки `teststatic.a`, которая содержит исходный код из файлов `lists.c` и `matrix.c`.

```
root@Guzovski:~/lab_c# gcc -c list.c matrix.c
root@Guzovski:~/lab_c# ar rcs teststatic.a list.o matrix.o
root@Guzovski:~/lab_c# ar t teststatic.a
list.o
matrix.o
root@Guzovski:~/lab_c#
```

7. Выполните компиляцию разделяемой библиотеки `testdynamic.so`, которая содержит исходный код из файлов `lists.c` и `matrix.c`.

```
root@Guzovski:~/lab_c# gcc -fPIC -shared list.c matrix.c -o testdynamic.so
```

8. Выполните компиляцию исполняемого файла в соответствии с пунктом 5 по очереди со следующими параметрами оптимизации: «-O1», «-O2», «-O3», «-Os». Сравните результаты работы и размер полученных файлов.

Результаты как и размеры одинаковы

```
root@Guzovski:~/lab_c# gcc main.c list.c matrix.c -o testfile_O1.out -lm -O1
root@Guzovski:~/lab_c# gcc main.c list.c matrix.c -o testfile_O2.out -lm -O2
root@Guzovski:~/lab_c# gcc main.c list.c matrix.c -o testfile_O3.out -lm -O3
root@Guzovski:~/lab_c# gcc main.c list.c matrix.c -o testfile_Os.out -lm -Os
root@Guzovski:~/lab_c# ls -lh testfile_*.out
-rwxr-xr-x 1 root root 17K Dec 11 21:29 testfile_O1.out
-rwxr-xr-x 1 root root 17K Dec 11 21:29 testfile_O2.out
-rwxr-xr-x 1 root root 17K Dec 11 21:29 testfile_O3.out
-rwxr-xr-x 1 root root 17K Dec 11 21:29 testfile_Os.out
```

9. Выполните компиляцию исполняемого файла в соответствии с пунктом 5 с включением всех предупреждений компилятора. Проанализируйте выдаваемые предупреждения.

```

root@Guzovski:~/lab_c# gcc main.c list.c matrix.c -o testfile_warn.out -lm -Wall
-Wextra -pedantic
main.c: In function 'main':
main.c:6:14: warning: unused parameter 'argc' [-Wunused-parameter]
    6 | int main(int argc, char* argv[])
      |             ^~~~~
main.c:6:26: warning: unused parameter 'argv' [-Wunused-parameter]
    6 | int main(int argc, char* argv[])
      |                   ^~~~~~
root@Guzovski:~/lab_c# █

```

10. Выполните компиляцию исполняемого файла в соответствии с пунктом 5 с включением в исполняемый файл отладочной информации и без оптимизации.

```

root@Guzovski:~/lab_c# gcc main.c list.c matrix.c -o testfile_dbg.out -lm -g -O0
root@Guzovski:~/lab_c# ls -lh testfile_dbg.out
-rwxr-xr-x 1 root root 20K Dec 11 21:37 testfile_dbg.out

```

Размер больше обычного

11. Выполним отладку файла, полученного в п. 10 с помощью отладчика gdb.

Запустите отладчик gdb одноименной командой. При этом на экране отобразится информация о версии программы и авторских правах, а также приглашение ко вводу встроенных команд:

```

$ gdb
GNU gdb (GDB) 7.4.1-debian
Copyright (C) 2012 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute
it.
There is NO WARRANTY, to the extent permitted by law. Type "show
copying"
and "show warranty" for details.
This GDB was configured as "i486-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
(gdb)

```

Загрузите файл для отладки следующей командой:

```

(gdb) file testfile.out
Reading symbols from /home/user/Labs/gcc/testfile.out...done.

```

```
(gdb) file testfile_dbg.out
Reading symbols from testfile_dbg.out...
(gdb) run
Starting program: /root/lab_c/testfile_dbg.out
```

Пример программы на языке C

Демонстрация работы со списком.

Создание списка.

Просмотр списка.

(1), (2), (3), (4), (5),

Удаление списка.

Запущен пример с матрицей.

0.000000.

[Inferior 1 (process 15908) exited normally]

(gdb) quit

A syntax error in expression, near `'.
(gdb) quit

root@Guzovski:~/lab_c# S

12. Просмотрите текст программы с помощью команды `list`:

```
(gdb) list
1      #include <stdlib.h>
2      #include "list.h"
3      #include "matrix.h"
4      int main( int argc, char* argv[] ){
5          printf("Пример программы на языке C\n\n");
```

```

6      ListFunction();
7      MatrixFunction();
8      return 0;
9  }

```

При этом на экране отобразится содержимое только основного файла main.c. В общем случае данная команда имеет следующий формат:

```
(gdb) list <файл><номер_строки>
```

В результате на экране отобразится 10 ближайших к указанной строке строк заданного файла. Например:

```

(gdb) list list.c:15
10
11 void AddNode( int NewValue ){
12     struct ListItem* Item;
13     Item = (struct ListItem*) malloc( sizeof(struct ListItem)
);
14     Item->Value = NewValue;
15     Item->Next = Root;
16     Root = Item;
17 }
18
19 void CreateList() {

```

Используя данную команду, отобразите весь исходный код файлов list.c и matrix.c.

```

type apropos word to search for commands related to word ...
Reading symbols from ./testfile_dbg.out...
(gdb) list list.c:1
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include "list.h"
4
5      struct ListItem {
6          int Value;
7          struct ListItem* Next;
8      };
9
10     struct ListItem* Root;
(gdb) list list.c:15
10     struct ListItem* Root;
11
12     void AddNode(int NewValue)
13     {
14         struct ListItem* Item;
15         Item = (struct ListItem*) malloc(sizeof(struct ListItem));
16         Item->Value = NewValue;
17         Item->Next = Root;
18         Root = Item;
19     }
(gdb)

```



```

(gdb) list matrix.c:1
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <math.h>
4      #include "matrix.h"
5
6      float ValMatr[30];
7
8      void MatrixFunction()
9      {
10         int I;
(gdb) list matrix.c:15
10         int I;
11         printf("Запущен пример с матрицей.\n");
12         for (I = 0; I < 29; I++) {
13             ValMatr[I] = 20 * sin(I / 10.0);
14         }
15
16         printf(" %f.\n", ValMatr[0]);
17     }

```

13. Перед запуском программы на выполнение, определим несколько точек останова.

Первую точку останова поставим на первый исполняемый оператор функции AddNode () в файле list.c:

```

(gdb) break list.c:13
Breakpoint 1 at 0x804851a: file list.c, line 13.

```

Второй поставим аппаратную точку останова при изменении переменной Root:

```

(gdb) watch Root
Hardware watchpoint 2: Root

```

Третью точку останова свяжем с запуском функции MatrixFunction():

```

(gdb) break MatrixFunction
Breakpoint 3 at 0x804866e: file matrix.c, line 8.

```

```

(gdb) break list.c:13
Breakpoint 1 at 0x11c6: file list.c, line 15.
(gdb) watch Root
Hardware watchpoint 2: Root
(gdb) break MatrixFunction
Breakpoint 3 at 0x133d: file matrix.c, line 11.
(gdb) info breakpoints

```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	0x000000000000011c6	in AddNode at list.c:15
2	hw watchpoint	keep	y		Root
3	breakpoint	keep	y	0x0000000000000133d	in MatrixFunction at matrix.c:11

```

(gdb)

```

14. Выполним пошаговую отладку программы с использованием определенных на предыдущем шаге выполнения точек останова.

Запустим программу на выполнение:

```
(gdb) run
Starting program: /root/lab_c/testfile_dbg.out
Пример программы на языке C
```

Демонстрация работы со списком.
Создание списка.

```
Breakpoint 1, AddNode (NewValue=5) at list.c:15
15      Item = (struct ListItem*) malloc(sizeof(struct ListItem));
(gdb)
```

```
(gdb) run
Starting program: /home/user/Labs/gcc/testfile.out
Пример программы на языке C
```

Демонстрация работы со списком.
Создание списка.

```
Breakpoint 1, AddNode (NewValue=5) at list.c:13
13      Item = (struct ListItem*) malloc( sizeof(struct ListItem)
);
```

Мы видим, что программа остановлена в первой точке останова при первом запуске функции AddNode(). Просмотрим значение переменных Root и Item.

```
(gdb) print Root
$1 = (struct ListItem *) 0x0
(gdb) print Item
$2 = (struct ListItem *) 0x0
$1 = (struct ListItem *) 0x0
(gdb) Print Item
Undefined command: "Print". Try "help".
(gdb) print Item
$2 = (struct ListItem *) 0x0
```

Переменная Root содержит нулевое значение (что для указателя означает NULL), а переменная Item еще не инициализирована. Выполним один оператор программы и еще раз посмотрим значение переменной Item.

```
(gdb) step
14      Item->Value = NewValue;
(gdb) print Item
$3 = (struct ListItem *) 0x804a008
$2 = (struct ListItem *) 0x0
(gdb) step
__GI___libc_malloc (bytes=16) at malloc.c:3031
```

Возобновим выполнение программы:

```
(gdb) continue
Continuing.
Hardware watchpoint 2: Root
```

```
Old value = (struct ListItem *) 0x0
New value = (struct ListItem *) 0x804a008
AddNode (NewValue=5) at list.c:17
```

```

(gdb) continue
Continuing.

Hardware watchpoint 2: Root

Old value = (struct ListItem *) 0x0
New value = (struct ListItem *) 0x5555555596b0
AddNode (NewValue=5) at list.c:19
19      }
17    }

```

Теперь мы остановились по аппаратной точке останова при изменении переменной Root. Остановка выполнения программы произошла в 17-й строке файла list.c (это функция AddNode()).

Просмотрим состояние точек останова и статистику по ним:

```

(gdb) info breakpoints
Num      Type             Disp Enb Address          What
1        breakpoint       keep y    0x0804851a in AddNode at
list.c:12
        breakpoint already hit 1 time
2        hw watchpoint    keep y    Root
        breakpoint already hit 1 time
3        breakpoint       keep y    0x0804866e in MatrixFunction at
matrix.c:8
(gdb) info breakpoint
Num      Type             Disp Enb Address          What
1        breakpoint       keep y    0x00005555555551c6 in AddNode at list.c:15
        breakpoint already hit 1 time
2        hw watchpoint    keep y    Root
        breakpoint already hit 1 time
3        breakpoint       keep y    0x000055555555533d in MatrixFunction
at matrix.c:11
. . .

```

Отключим две первые точки останова и опять посмотрим состояние точек останова:

```

(gdb) disable breakpoint 1
(gdb) disable breakpoint 2
(gdb) info breakpoints
Num      Type             Disp Enb Address          What
1        breakpoint       keep n    0x0804851a in AddNode at
list.c:12

```

```

    breakpoint already hit 1 time
2      hw watchpoint keep n          Root
    breakpoint already hit 1 time
3      breakpoint keep y 0x0804866e in MatrixFunction at
matrix.c:8
(gdb) disable breakpoint 1
(gdb) disable breakpoint 2
(gdb) info breakpoint
Num      Type          Disp Enb Address          What
1        breakpoint    keep n  0x00005555555551c6 in AddNode at list.c:15
    breakpoint already hit 1 time
2        hw watchpoint keep n          Root
    breakpoint already hit 1 time
3        breakpoint    keep y  0x000055555555533d in MatrixFunction
    at matrix.c:11

```

Продолжим выполнение программы:

```

(gdb) continue
Continuing.
Просмотр списка.
(1), (2), (3), (4), (5),
Удаление списка.

```

```

Breakpoint 3, MatrixFunction () at matrix.c:8
8      printf("Запущен пример с матрицей.\n");

```

```

(gdb) continue
Continuing.
Просмотр списка.
(1), (2), (3), (4), (5),
Удаление списка.

Breakpoint 3, MatrixFunction () at matrix.c:11
11      printf("Запущен пример с матрицей.\n");

```

Теперь мы остановились в третьей точке останова в функции MatrixFunction. Продемонстрируем возможности отладчика по работе с переменными. Посмотрим, а потом изменим значение нулевого элемента массива ValMatr:

```

(gdb) print ValMatr[0]
$8 = 0
(gdb) set var ValMatr[0]=5
(gdb) print ValMatr[0]
$9 = 5

```

```

(gdb) print ValMatr[0]
$3 = 0
(gdb) set ValMatr[0]=5
(gdb) print ValMatr[0]
$4 = 5
(gdb)

```

Продолжим выполнение программы:

```
(gdb) continue
Continuing.
Запущен пример с матрицей.
0.
[Inferior 1 (process 3553) exited normally]
```

```
(gdb) continue
Continuing.
Запущен пример с матрицей.
0.000000.
[Inferior 1 (process 15980) exited normally]
(gdb) █
```

Как видно из последнего листинга, выполнение программы завершено. Выйдем из отладчика следующей командой:

```
(gdb) quit
```

15. Далее выполним отладку с помощью отладчика Data Display Debugger. Данный отладчик реализует более привычную среду отладки с графическим интерфейсом. Также он обладает достаточно мощными средствами отображения отладочных данных. Запустим отладчик следующей командой:

```
$ ddd
```

В результате на экране появится окно отладчика. В меню «File» выберите пункт «Open Program». В появившемся диалоговом окне выберите наш исполняемый файл `testfile.out` и нажмите кнопку «Open». В результате окно отладчика приобретет вид, показанный на рис. 7.1.

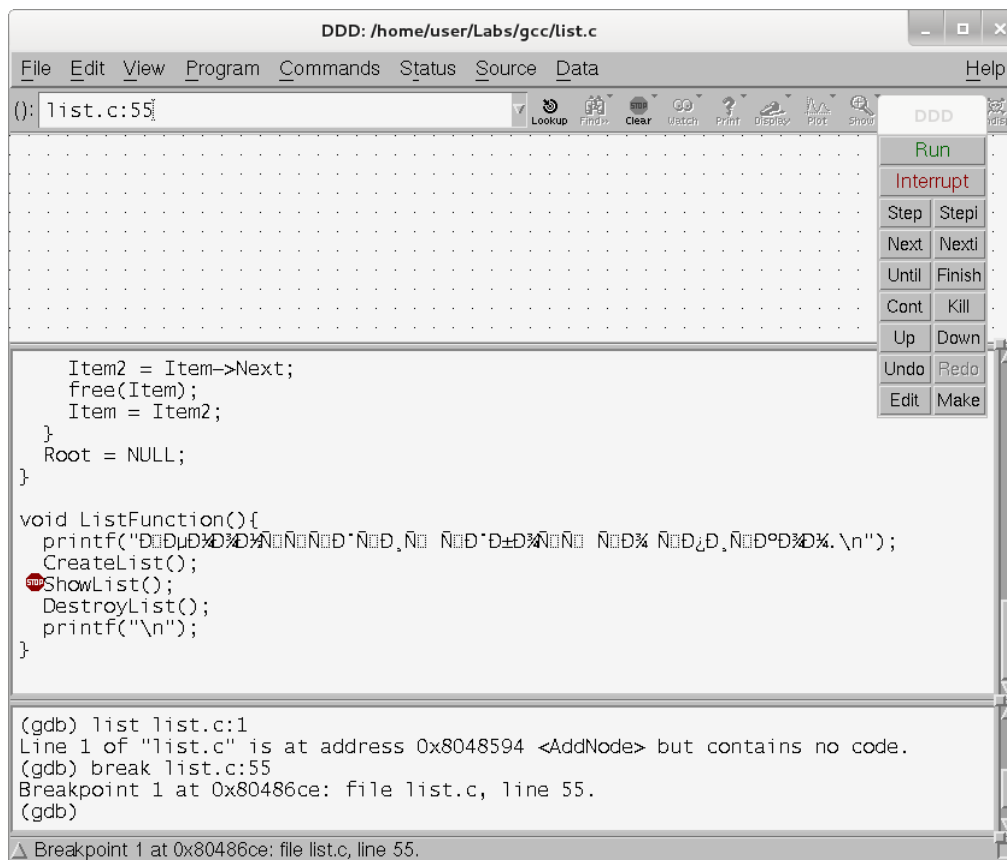


Рис. 7.2. Изображение окна отладчика ddd с установленной точкой останова

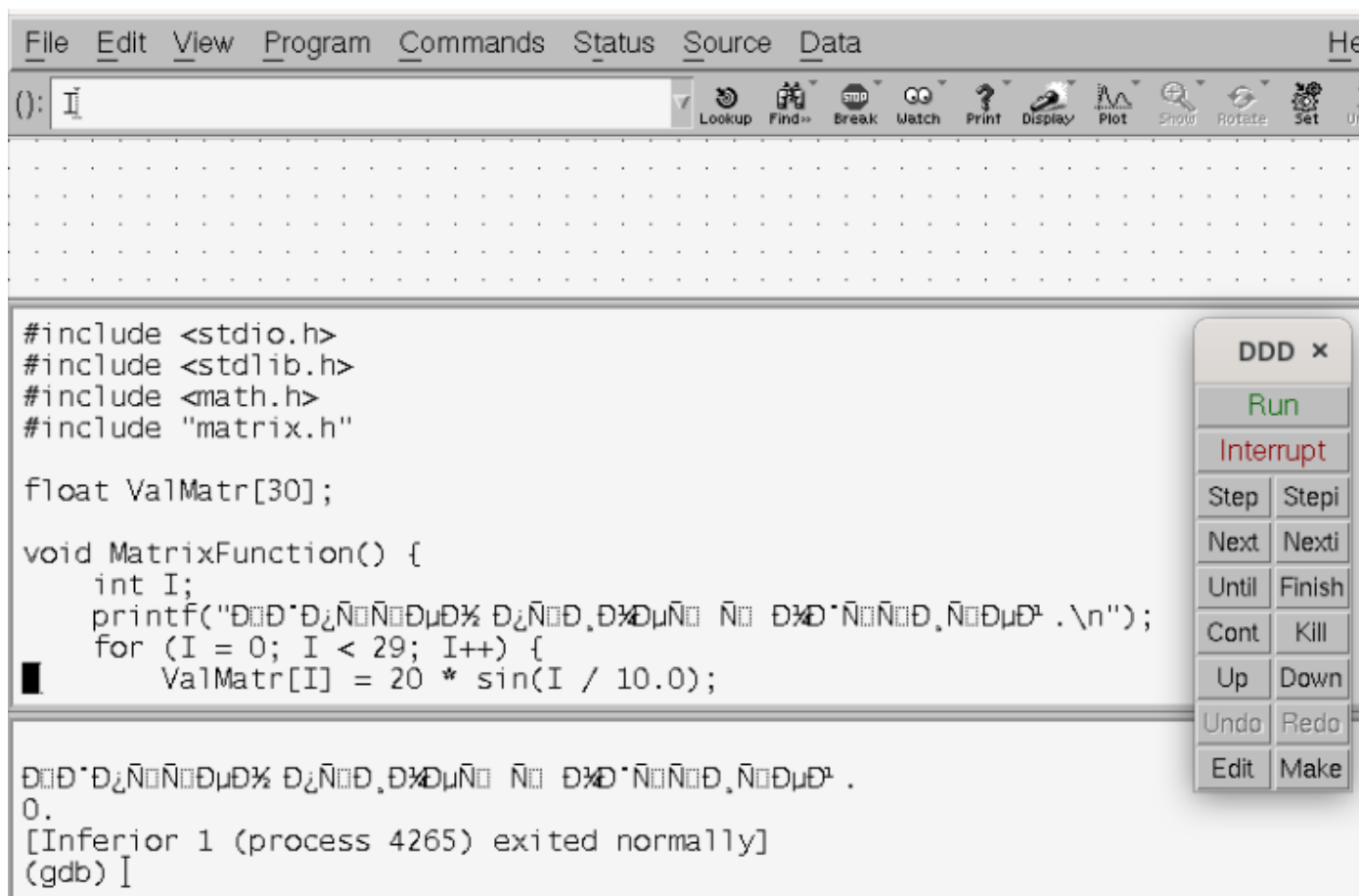
Откройте файл исходного кода matrix.c и поставьте точку останова в функции MatrixFunction() внутри цикла.

Запустите программу, нажав на кнопку «Run» в окне быстрого выбора команд. В результате выполнение программы будет остановлено при достижении первой точки останова. Нажимайте кнопку «Step» пока не войдете в функцию, и переменной Item не будет присвоено значение.

Продемонстрируем возможности отладчика по изображению списков. В области кода щелкните правой кнопкой мыши на названии переменной Item. В появившемся контекстном меню выберите «Display Item». При этом в области отображения данных появится прямоугольник, соответствующий данной переменной, в верхней части которого содержится имя переменной, а в нижней – значение адреса, на который ссылается указатель Item. В области отображения данных щелкните правой кнопкой мыши на данном значении адреса и в появившемся контекстном меню выберите пункт «Display *()». В результате в области отображения данных появится изображение первого элемента списка, связанное стрелкой с изображением переменной Item. Действуя аналогичным образом, можно получить изображение всего списка в виде диаграммы (рис. 7.3).

области отображения данных. Как видно, массив отображается в виде таблицы значений, что достаточно удобно (рис. 7.4).

В окне быстрого выбора команд нажмите кнопку «Continue». Работа программы завершится. Закройте окно отладчика ddd.



КОНТРОЛЬНЫЕ ВОПРОСЫ

Ответы на контрольные вопросы:

1. Что собой представляет компилятор GCC? Какие основные аппаратные платформы и языки программирования им поддерживаются?

GCC (GNU Compiler Collection) — это набор компиляторов с открытым исходным кодом, разработанный проектом GNU. Поддерживает множество языков: C, C++, Fortran, Ada, Go, D и др.

Поддерживаемые архитектуры: x86, x86-64, ARM, PowerPC, MIPS, SPARC и др. GCC кроссплатформенный и работает на большинстве Unix-подобных систем, включая Linux.

2. Что собой представляет процесс компиляции программы на языке C компилятором GCC? Каковы основные этапы данного процесса? Какие получаются промежуточные и конечные результаты?

Процесс компиляции GCC включает 4 этапа:

Преппроцессинг (cpp) — обработка директив #include, #define и т.д. → .i файл.

Компиляция — перевод в ассемблерный код → .s файл.

Ассемблирование — преобразование ассемблера в объектный код → .o файл.

Компоновка (линковка) — объединение объектных файлов и библиотек в исполняемый файл.

Конечный результат — исполняемый файл (например, a.out или testfile.out).

3. С помощью какой команды выполняется компиляция простейшей программы на языке C? Как выполняется компиляция программы, состоящей из нескольких файлов? Как выполняется компиляция программы, использующей статическую библиотеку? Как выполняется компиляция статической и динамической библиотеки?

Простейшая программа:

- gcc program.c -o program

Несколько файлов:

- gcc main.c file1.c file2.c -o program

Со статической библиотекой libmy.a:

- gcc main.c -L. -lmy -o program

Создание статической библиотеки:

- gcc -c file1.c file2.c
- ar rcs libmy.a file1.o file2.o

Создание динамической библиотеки:

- gcc -fPIC -c file1.c file2.c
- gcc -shared -o libmy.so file1.o file2.o

4. Как при компиляции задать используемый стандарт языка C? Как включить в исполняемый файл отладочную информацию? Какие существуют режимы оптимизации кода в компиляторе GCC? Какие существуют параметры выдачи предупреждений компилятора?

Стандарт C:

-std=c90, -std=c99, -std=c11, -std=c17

Отладочная информация:

-g

Оптимизация:

-O0 (без оптимизации), -O1, -O2, -O3, -Os (оптимизация по размеру), -Ofast

Предупреждения:

-Wall (основные), -Wextra (дополнительные), -Werror (предупреждения как ошибки)

5. Из каких структурных частей состоит компилятор GCC? Какие в нем существуют промежуточные формы представления программы?

GCC состоит из:

фронтендов (для каждого языка),
среднего слоя (GENERIC, GIMPLE — промежуточные представления),
оптимизатора,
бэкендов (для каждой архитектуры).
Промежуточные формы:

GENERIC — высокоуровневое дерево,
GIMPLE — упрощённое трёхадресное представление,
RTL (Register Transfer Language) — низкоуровневое представление, близкое к машинному коду.

6. Что собой представляет отладчик GDB? Перечислите основные возможности и основные встроенные команды данного отладчика.

GDB (GNU Debugger) — консольный отладчик для программ на C/C++ и других языках.

Возможности:

Установка точек останова (break)
Пошаговое выполнение (step, next)
Просмотр и изменение переменных (print, set)
Наблюдение за изменениями (watch)
Анализ стека вызовов (backtrace)
Загрузка исполняемых файлов (file)
Основные команды:

run — запуск
break <file:line> — точка останова
watch <var> — наблюдение за переменной
step / next — шаг с/без входа в функцию
print <var> — вывод значения
continue — продолжить выполнение
info breakpoints — список точек останова
quit — выход

7. Что собой представляет отладчик DDD? Каким образом он связан с отладчиком GDB? Каковы основные достоинства отладчика DDD?

DDD (Data Display Debugger) — графический фронтенд для GDB (а также для других отладчиков,

например, DBX).

Связь с GDB:

DDD использует GDB как движок отладки, передавая ему команды и отображая результаты визуально.

Достоинства:

Графический интерфейс (удобнее для новичков)

Визуализация структур данных (списков, деревьев, массивов)

Возможность отображать данные в виде диаграмм

Поддержка редактирования исходного кода

Интерактивное управление точками останова